Midterm Review

CSCI 235 through DFS and BFS

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 13 Thursday, October 15, 2026

Today and the Midterm

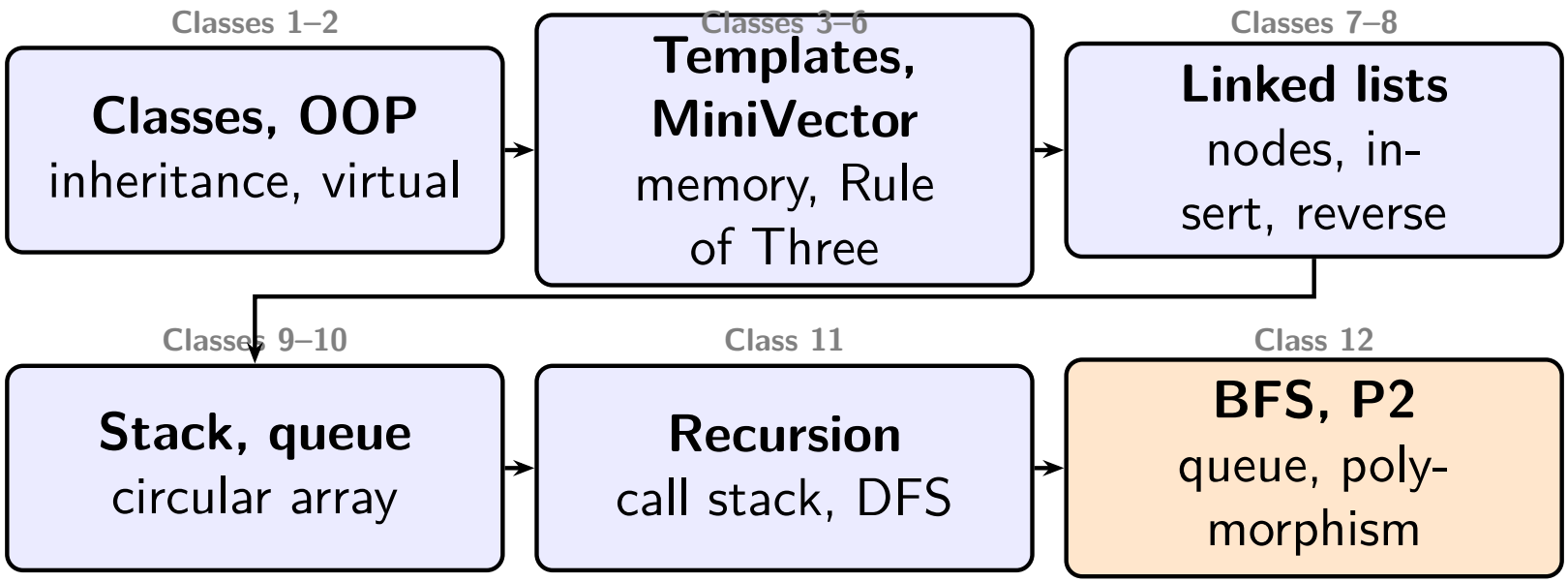
The midterm

Tuesday, October 27, in class, 8:30–9:45 AM. It covers **P1 MiniVector**, **P2 Maze Solver**, and all course material **through DFS and BFS**.

- **Not on the midterm:** state-space search (Oct 20) and permutations (Oct 22). They lead into Project 3.
- **Valgrind** is taught as professional practice and graded through your projects and exercises. It is not tested.
- Today: for each of seven topics, a short recap, then practice problems. **Try each problem first**; the answer appears on the next page of the same slide.

Seven topics: classes and polymorphism; templates and MiniVector; linked lists; stacks and queues; recursion; DFS and BFS; Big-O.

The Course So Far



Everything on this chart is fair game. Projects: **P1** MiniVector (memory, templates), **P2** Maze Solver (DFS, BFS, polymorphism). Exercises 1–7 repeat the same ideas in small pieces.

Topic 1: Classes, Inheritance, and Polymorphism

Classes

- Private data, public interface (**encapsulation**); a constructor establishes the **invariant**.
- **Copy constructor**: builds a new object from an existing one (`Rectangle b = a;`).
- **Copy assignment**: overwrites an object that already exists (`b = a;`). It must **avoid leaking the old object's memory** and **return `*this`** (the current object).

Inheritance and `virtual`

- Derived *is-a* base; `protected` is visible to derived classes.
- A base pointer or reference can refer to a derived object.
- `virtual` $\Rightarrow$ the **real object** decides at run time. Without it, the **pointer type** decides at compile time.
- `= 0`: pure virtual, so the class is **abstract**.
- If an object may be deleted through a base pointer, the base class needs a **virtual destructor**.

Practice 1: What Prints?

```
struct Shape {
    std::string name() const { return "Shape"; } // NOT virtual
    virtual double area() const = 0;
    virtual ~Shape() {}
};
struct Circle : Shape {
    std::string name() const { return "Circle"; }
    double area() const override { return 3.0; }
};
Shape* p = new Circle;
std::cout << p->name() << p->area(); // what prints?
```

(A) Circle3

(B) Shape3

(C) Circle

(D) It does not compile.

Practice 1: What Prints?

```
struct Shape {
    std::string name() const { return "Shape"; } // NOT virtual
    virtual double area() const = 0;
    virtual ~Shape() {}
};
struct Circle : Shape {
    std::string name() const { return "Circle"; }
    double area() const override { return 3.0; }
};
Shape* p = new Circle;
std::cout << p->name() << p->area(); // what prints?
```

(A) `Circle3`

(B) `Shape3`

(C) `Circle`

(D) It does not compile.

Answer: (B). `name()` is not virtual: the pointer type picks `Shape::name`. `area()` is virtual: the real object picks `Circle::area`.

Practice 1b: Pointer, Reference, or Copy?

```
struct Shape {
    virtual std::string name() const { return "Shape"; }
    virtual ~Shape() {}
};
struct Circle : Shape {
    std::string name() const override { return "Circle"; }
};
Circle c;
Shape* p = &c; Shape& r = c; Shape s = c;
std::cout << p->name() << r.name() << s.name();
```

(A) CircleCircleCircle

(B) CircleCircleShape

(C) ShapeShapeShape

(D) CircleShapeShape

Practice 1b: Pointer, Reference, or Copy?

```
struct Shape {
    virtual std::string name() const { return "Shape"; }
    virtual ~Shape() {}
};
struct Circle : Shape {
    std::string name() const override { return "Circle"; }
};
Circle c;
Shape* p = &c; Shape& r = c; Shape s = c;
std::cout << p->name() << r.name() << s.name();
```

(A) CircleCircleCircle

(B) **CircleCircleShape**

(C) ShapeShapeShape

(D) CircleShapeShape

Answer: (B). Through `p` and `r` the real object (`Circle`) decides. `Shape s = c;` **copies** only the `Shape` part (*slicing*), so `s` is a plain `Shape`. (This is *overriding*, not *overloading*.)

Practice 2: Which Function Runs?

Assume a class `Rectangle` with a default constructor, a constructor `Rectangle(int,int)`, a copy constructor, and a copy assignment operator. For each line, which one runs?

Line	Runs
`Rectangle a(2, 3);`	
`Rectangle b = a;`	
`Rectangle c;`	
`c = a;`	
`Rectangle d(a);`	
`void f(Rectangle r); f(a);`	

Practice 2: Which Function Runs?

Assume a class `Rectangle` with a default constructor, a constructor `Rectangle(int,int)`, a copy constructor, and a copy assignment operator. For each line, which one runs?

Line	Runs
`Rectangle a(2, 3);`	**constructor with two ints**
`Rectangle b = a;`	**copy constructor** (a new object is being created)
`Rectangle c;`	**default constructor**
`c = a;`	**copy assignment** (c already exists)
`Rectangle d(a);`	**copy constructor**
`void f(Rectangle r); f(a);`	**copy constructor** (pass by value copies)

Rule of thumb: a new object is built with a constructor; an existing object is overwritten by assignment.

Topic 2: Templates, MiniVector, and Memory

- `template <typename T>`: one definition, many types, fixed at compile time.
- **Stack** memory is automatic; **heap** memory from `new` must be released: `delete` for one object, `delete[]` for an array.
- **Growth:** when full, allocate a bigger array (double the capacity), copy, `delete[]` the old one. `push_back` is **amortized** $O(1)$.
- **Rule of Three:** if a class owns memory, write the **destructor**, the **copy constructor**, and the **copy assignment**.

Deep copy and assignment

```
MiniVector(const MiniVector& o)
  : data_(new T[o.capacity_]),
    size_(o.size_),
    capacity_(o.capacity_) {
  for (size_t i = 0; i < size_; ++i)
    data_[i] = o.data_[i];
}
MiniVector& operator=(const MiniVector& o) {
  if (this != &o) {          // guard
    T* fresh = new T[o.capacity_];
    for (size_t i = 0; i < o.size_; ++i)
      fresh[i] = o.data_[i];
    delete[] data_;          // free old
    data_ = fresh;
    size_ = o.size_;
    capacity_ = o.capacity_;
  }
  return *this;
}
```

Practice 3(a): Capacity and Copies

(a) A `MiniVector` starts with capacity 1 and **doubles** when it is full. Nine values are pushed. Fill in the capacity *after* each push, and count the elements copied during resizes.

push #	1	2	3	4	5	6	7	8	9
capacity									
copied									

Practice 3(a): Capacity and Copies

(a) A `MiniVector` starts with capacity 1 and **doubles** when it is full. Nine values are pushed. Fill in the capacity *after* each push, and count the elements copied during resizes.

push #	1	2	3	4	5	6	7	8	9
capacity	1	2	4	4	8	8	8	8	16
copied	0	1	2	0	4	0	0	0	8

Starting from capacity 1, resizes happen at pushes 2, 3, 5, 9, copying $1 + 2 + 4 + 8 = \mathbf{15}$ elements, fewer than 2×9: that is why the average cost per push stays $O(1)$.

Elements copied at each resize: new capacity 2 $\rightarrow$ copy 1; capacity 4 $\rightarrow$ copy 2; capacity 8 $\rightarrow$ copy 4; capacity 16 $\rightarrow$ copy 8.

Practice 3(b): A Bug in a Copy Constructor

(b) A copy constructor is written as `MiniVector(const MiniVector& o) : data_(o.data_), size_(o.size_), capacity_(o.capacity_) {}`. What goes wrong?

(A) Nothing; it is a correct copy.

(B) Both objects share one array, so the array is deleted twice.

(C) It does not compile.

(D) It copies too slowly.

Practice 3(b): A Bug in a Copy Constructor

(b) A copy constructor is written as `MiniVector(const MiniVector& o) : data_(o.data_), size_(o.size_), capacity_(o.capacity_) {}`. What goes wrong?

(A) Nothing; it is a correct copy.

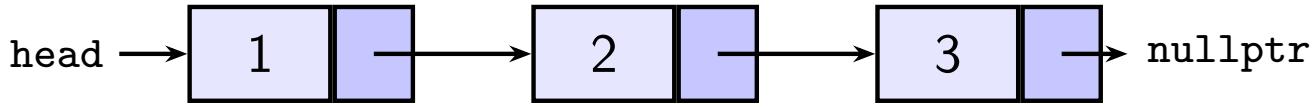
(B) **Both objects share one array, so the array is deleted twice.**

(C) It does not compile.

(D) It copies too slowly.

Answer: (B). This is a *shallow* copy: it copies the pointer, not the array. Changing one object changes the other, and the two destructors both `delete[]` the same memory.

Topic 3: Singly Linked Lists



- `struct Node { int data; Node* next; };` and a `head` pointer. Empty list: `head == nullptr`.
- **Traverse:** `for (Node* curr = head; curr != nullptr; curr = curr->next)`.
- **Prepend:** `Node* n = new Node{v, head}; head = n;` (new node first points at the old head, *then* head moves).
- **Delete a node:** save what you need *before* `delete`; handle head, middle, tail, and missing value.
- **Reverse in place:** three pointers, `prev`, `curr`, `next`: save next, flip the arrow, step forward.

Practice 4: Reverse a List

```
Node* prev = nullptr;
Node* curr = head;                  // list: 1 -> 2 -> 3
while (curr != nullptr) {
    Node* next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
head = prev;
```

Fill in the table after each iteration. Final list?

After	`prev`	`curr`	**flipped node's** `next`
before loop	`nullptr`	1	–
iteration 1			
iteration 2			
iteration 3			

Practice 4: Reverse a List

```
Node* prev = nullptr;
Node* curr = head;                  // list: 1 -> 2 -> 3
while (curr != nullptr) {
    Node* next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
head = prev;
```

Fill in the table after each iteration. Final list?

After	`prev`	`curr`	**flipped node's `next`**
before loop	`nullptr`	1	–
iteration 1	1	2	$1 \to$ `nullptr`
iteration 2	2	3	$2 \to 1$
iteration 3	3	`nullptr`	$3 \to 2$

After the loop `head = prev` points to 3: the list is $\mathbf{3} \to \mathbf{2} \to \mathbf{1}$. We save `next` first because `curr->next = prev` would otherwise lose the rest of the list.

Practice 5: Spot the Bug

```
void popFront() {
    delete head;
    head = head->next;
}
```

What is wrong?

(A) It leaks the removed node.

(B) It reads `head->next` after the node was deleted.

(C) It does not compile.

(D) Nothing; it is correct.

Practice 5: Spot the Bug

```
void popFront() {
    delete head;
    head = head->next;
}
```

What is wrong?

(A) It leaks the removed node.

(B) **It reads `head->next` after the node was deleted.**

(C) It does not compile.

(D) Nothing; it is correct.

Answer: (B). After `delete head`, the node's memory is gone, so `head->next` is a **use after free**. It also crashes on an empty list. Fix:

```
void popFront() {
    if (head == nullptr) { return; }
    Node* toDelete = head;    // 1. remember the node
    head = head->next;        // 2. move head first
    delete toDelete;          // 3. then free it
}
```

Topic 4: Stacks and Queues

Stack: last in, first out

- `push`, `pop`, `top`, `empty`.
- Array version: `top` index; linked version: head pointer.
- The **call stack** is a stack of function frames.

Queue: first in, first out

- `enqueue` at the back, `dequeue` from the front.
- `front_` is the **first** item; `back_` is the **last** item (not one past).

Circular array, capacity C

- Empty: `front_ = back_ = -1`, size 0.
- Move an index forward with `(i + 1) % C`.
- Not wrapped (`front_ <= back_`): size = `back_ - front_ + 1`.
- Wrapped (`back_ < front_`): size = `back_ - front_ + 1 + C`.
- Linked version: `front_` and `back_` are node pointers, so both operations are $O(1)$.

Practice 6: Trace a Circular Queue

Capacity $C = 4$. Start empty. Fill in `front_`, `back_`, and the size after each operation.

Operation	`front_`	`back_`	**size**
enqueue A			
enqueue B			
enqueue C			
dequeue			
enqueue D			
enqueue E			
dequeue			

Practice 6: Trace a Circular Queue

Capacity $C = 4$. Start empty. Fill in `front_`, `back_`, and the size after each operation.

Operation	`front_`	`back_`	**size**
enqueue A	0	0	1
enqueue B	0	1	2
enqueue C	0	2	3
dequeue	1	2	2
enqueue D	1	3	3
enqueue E	1	0 (wrapped)	$0 - 1 + 1 + 4 = 4$
dequeue	2	0	$0 - 2 + 1 + 4 = 3$

The queue now holds **C, D, E** (front first), stored at array indexes 2, 3, 0. **Stack check:** push 1, 2, 3, pop, push 4, pop, pop pops **3, 4, 2** and leaves **1**.

Topic 5: Recursion and the Call Stack

- Every recursive function needs a **base case** (answer immediately) and a **smaller problem** of the same kind.
- **Leap of faith:** assume the recursive call is correct; what is left to do?
- Each call gets a **stack frame**: a new call pushes one, a return pops one. Too many frames $\Rightarrow$ stack overflow.
- **Print before the call:** forward order. **Print after the call:** reverse order.
- **Recursive DFS on a grid:** mark the cell visited, try neighbors in the stated order, return success or failure. Visited marks stay; frames pop.

Two helpers worth remembering

`digitSum(n) = n % 10 + digitSum(n / 10)`, base case `n < 10`. A **helper** carries extra parameters (an index, a `visited` table by non-`const` reference) so the public function stays simple.

Practice 7: Trace the Calls

```
int f(int n) {
    if (n == 0) { return 0; }
    return n + f(n - 1);
}
```

(a) What does `f(4)` return?

```
void show(int n) {
    if (n == 0) { return; }
    std::cout << n;
    show(n - 1);
    std::cout << n;
}
```

(b) What does `show(3)` print?

Practice 7: Trace the Calls

```
int f(int n) {
    if (n == 0) { return 0; }
    return n + f(n - 1);
}
```

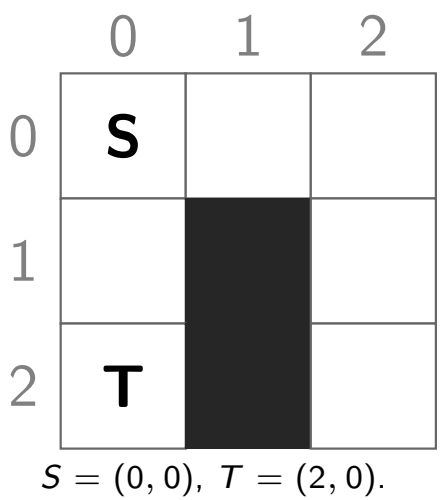
```
void show(int n) {
    if (n == 0) { return; }
    std::cout << n;
    show(n - 1);
    std::cout << n;
}
```

(a) What does `f(4)` return? **(b)** What does `show(3)` print?

(a) $f(4) = 4 + f(3) = 4 + 3 + f(2) = 4 + 3 + 2 + f(1) = 4 + 3 + 2 + 1 + f(0) =$ **10**. The additions happen on the way *back up*.

(b) `show(3)` prints **3**, calls `show(2)` (prints 2, calls `show(1)`: prints 1, calls `show(0)`, which returns); then each call prints its number again on the way back: **3 2 1 1 2 3**.

Practice 8: Recursive DFS on a Maze

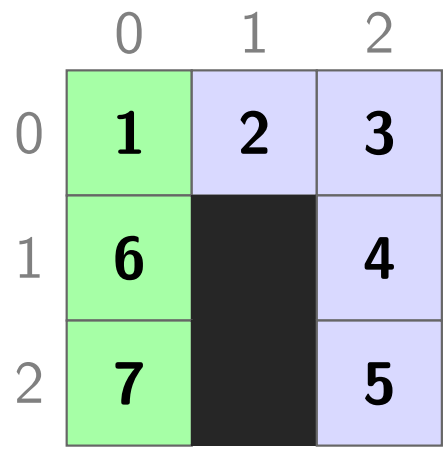


$S = (0, 0)$, $T = (2, 0)$.

Neighbor order: **left, right, up, down**.
Each call marks its cell visited first.

(a) In what order are the cells entered?
(b) What is the final route, and why are the cells of the first branch not on it?

Practice 8: Recursive DFS on a Maze



Order of calls; green = route.

Neighbor order: **left, right, up, down**. Each call marks its cell visited first.

(a) In what order are the cells entered?
(b) What is the final route, and why are the cells of the first branch not on it?

(a)
$(0,0), (0,1), (0,2), (1,2), (2,2), (1,0), (2,0)$: seven calls.

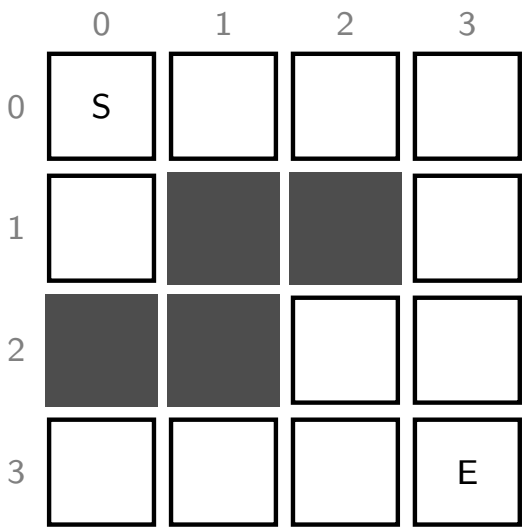
(b) Route $(0,0) \to (1,0) \to (2,0)$. DFS tries *right* before *down*; the right-hand branch ends at the dead end $(2,2)$ and returns empty-handed. Its cells were *visited*, but no call below them reached T, so they are **not on the route**.

Topic 6: DFS, BFS, and Polymorphism in P2

	DFS	BFS
Container	Stack (the call stack, or an explicit one)	Queue (`std::queue<Coord>`)
Idea	Go deep along one branch, then back up	Explore layers: distance 1, 2, 3, ...
Route	The first route found; **not** necessarily shortest	A route with the **fewest cells**
Marking	Mark when the call **enters** the cell	Mark when the cell is **enqueued**
Extra data	`visited` table	`visited` and a `parent` table of `Coord`
Route output	Cells pushed on an answer stack as calls return	Follow parents from end to start; a stack puts the start on top

Polymorphism: `Maze_Solver` declares `virtual getPath() const = 0`; `DFS_Solver` and `BFS_Solver` override it. Through a `Maze_Solver*`, the **real object** chooses the version. `reachable()` is written once in the base class and calls `getPath()`.

Practice 9: BFS on the Project 2 Maze



$S = (0, 0)$, $E = (3, 3)$.

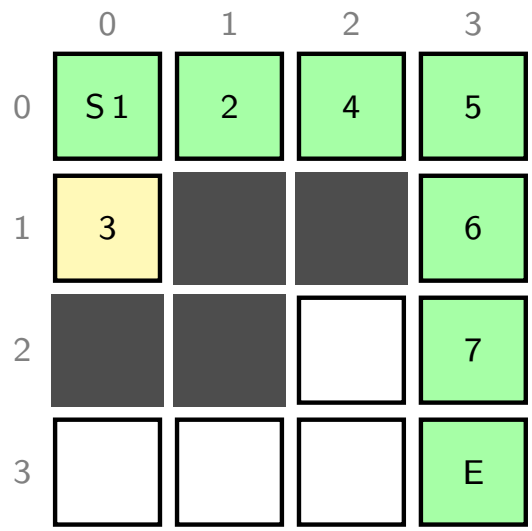
Neighbor order **left, right, up, down**; stop when E is discovered.

(a) What is in the queue right after $(0, 1)$ has been dequeued and its neighbors processed?

(b) How many cells are on the BFS route? How many on DFS's route?

(c) In what order does BFS dequeue cells?

Practice 9: BFS on the Project 2 Maze



Order of dequeuing; green = route.

Neighbor order **left, right, up, down**; stop when E is discovered.

(a) What is in the queue right after $(0, 1)$ has been dequeued and its neighbors processed?
(b) How many cells are on the BFS route? How many on DFS's route?
(c) In what order does BFS dequeue cells?

(a) $(1, 0)$, $(0, 2)$ (front first).
(b) BFS: **7** cells. DFS (same order): **9** cells, after entering 11.
(c)
$(0, 0), (0, 1), (1, 0), (0, 2), (0, 3), (1, 3), (2, 3)$; E is discovered while $(2, 3)$ is processed. BFS found the shorter route.

Practice 10: Which `getPath` Runs?

```
BFS_Solver bfs(room, start, end);
Maze_Solver* p = &bfs;
auto path = p->getPath();
bool ok = p->reachable();    // reachable() is NOT virtual
```

Which statement is true?

(A) `getPath()` runs the `Maze_Solver` version.

(B) `getPath()` runs the `BFS_Solver` version, and so does the `getPath()` call inside `reachable()`.

(C) The code does not compile because `Maze_Solver` is abstract.

(D) `reachable()` runs the `BFS_Solver` version of `reachable()`.

Practice 10: Which `getPath` Runs?

```
BFS_Solver bfs(room, start, end);
Maze_Solver* p = &bfs;
auto path = p->getPath();
bool ok = p->reachable();    // reachable() is NOT virtual
```

Which statement is true?

(A) `getPath()` runs the `Maze_Solver` version.

(B) **`getPath()` runs the `BFS_Solver` version, and so does the `getPath()` call inside `reachable()`.**

(C) The code does not compile because `Maze_Solver` is abstract.

(D) `reachable()` runs the `BFS_Solver` version of `reachable()`.

Answer: (B). You may *point to* an abstract class; you just cannot create one. `getPath()` is virtual, so the real object decides. `reachable()` is not virtual: the base version runs, but the `getPath()` call inside it is virtual, so it still reaches BFS's version.

Topic 7: Big-O

Growth	Typical source
$O(1)$	index into an array; push on a stack
$O(\log n)$	halving the problem each step
$O(n)$	one pass; traverse a list; BFS/DFS on n cells
$O(n^2)$	two nested passes over n

- Count **how many times the busiest line runs** as n grows; drop constants and smaller terms.
- Sequential loops **add**; nested loops **multiply**.
- **Amortized $O(1)$:** one expensive resize is paid for by many cheap pushes.
- DFS and BFS each enter or enqueue every cell at most once: $O(n)$ for n cells.

Practice 11: What Is the Running Time?

Code	Time
`for (i = 0; i < n; ++i) for (j = 0; j < n; ++j) ++count;`	
`for (i = 1; i < n; i *= 2) ++count;`	
`for (i = 0; i < n; ++i) ++a; for (j = 0; j < n; ++j) ++b;`	
`for (i = 0; i < n; ++i) for (j = 0; j < i; ++j) ++count;`	
n calls to `push_back` on a doubling `MiniVector`	
Recursive `digitSum(n)` for a number with *d* digits	

Practice 11: What Is the Running Time?

Code	Time
`for (i = 0; i < n; ++i) for (j = 0; j < n; ++j) ++count;`	$O(n^2)$
`for (i = 1; i < n; i *= 2) ++count;`	$O(\log n)$
`for (i = 0; i < n; ++i) ++a; for (j = 0; j < n; ++j) ++b;`	$O(n)$ (loops add)
`for (i = 0; i < n; ++i) for (j = 0; j < i; ++j) ++count;`	$O(n^2)$ $(0 + 1 + \cdots + (n-1))$
n calls to `push_back` on a doubling `MiniVector`	$O(n)$ total, $O(1)$ amortized each
Recursive `digitSum(n)` for a number with d digits	$O(d)$: one call per digit

Common Mistakes

- Shallow copy of an owning pointer: two objects, one array, double delete.
- `delete` instead of `delete[]` for an array; deleting a derived object through a base pointer without a virtual destructor.
- Using a node after `delete`; moving `head` too late.
- Off-by-one in circular indexes; forgetting `% C`.
- Calling a non-virtual function through a base pointer and expecting the derived version.
- Recursion with no base case, or a call that does not get closer to it.
- Printing *before* vs. *after* the recursive call: forward vs. reverse.
- Confusing **visited** with **on the route**.
- BFS: marking cells when *dequeued* instead of *enqueued* (duplicates in the queue).
- Using $(0, 0)$ as the "no parent" marker; $(-1, -1)$ is safe.

How to Study for Tuesday

1. **Redo, don't reread.** Cover the answers of Exercises 1–7 and re-trace each one on paper.
2. **Draw.** Pointer pictures for lists, array pictures for the circular queue, call stacks for recursion, queue and parent tables for BFS.
3. **Use the course materials:** the lecture slides and Q&A decks (Classes 10 and 11), the flashcards, the Queue and Recursion Card Quest games, and the LeetCode lists on stacks, queues, and recursion.
4. **Re-read your own project code:** P1 MiniVector and P2 Maze Solver. Be able to explain every line you wrote.
5. **Bring** your ID and a pen or pencil. Arrive early: the exam starts at 8:30.

Project 2

Due **Sunday, October 25, 11:59 PM**, two days before the exam. Submit early; then use the weekend for review.

Summary

- **Midterm:** Tuesday, October 27, 8:30–9:45 AM; through DFS and BFS; P1 and P2 included.
- Seven topics: classes and polymorphism; templates and MiniVector; linked lists; stacks and queues; recursion; DFS and BFS; Big-O.
- **Trace, don't guess:** draw the call stack, the pointers, the queue.
- Next class (Oct 20): state-space search. It leads into Project 3 and is **not** on the midterm.

Questions?